chnclsr / sybil-ui

Type / to search

Code Issues Pull requests Actions Projects Security and quality Insights Settings

sybil-ui Private

Watch 0 Fork 0 Star 1

About

BT görüntüleri inceleme ve etiketleme arayüzü

Readme
Activity
1 star
0 watching
0 forks

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors 2

chnclsr devC
aokirii Kasım

Languages

JavaScript 47.1% Python 35.4% HTML 17.5%

frontend-b... 4 Branches 0 Tags Go to file T Go to file Add file Code

This branch is 42 commits ahead of and 107 commits behind `gelistirme` . Contribute

Cihan chore: remove AGENTS.md from tracking and ignore in .gitignore c058151 · 2 weeks ago 42 Commits

.github	feat(diagrams): set dark mode as default an...	3 weeks ago
archive	feat(viewer-migration-archive): DICOM 512...	last month
database	feat(viewer-migration-archive): DICOM 512...	last month
docs	chore(docs): remove office files (docx, xlsx) ...	2 weeks ago
frontend	style: remove Cezeri logo image assets com...	3 weeks ago
scripts	feat(docker): add production dockerization,...	3 weeks ago
src	feat(docker): add production dockerization,...	3 weeks ago
test	feat(docker): add production dockerization,...	3 weeks ago
web/static	style: remove Cezeri logo image assets com...	3 weeks ago
.dockerignore	feat(docker): add production dockerization,...	3 weeks ago
.env.docker.example	feat(docker): add production dockerization,...	3 weeks ago
.env.example	Initial frontend backend merge	last month
.gitignore	chore: remove AGENTS.md from tracking a...	2 weeks ago
	feat(docker): add production dockerization,...	3 weeks ago

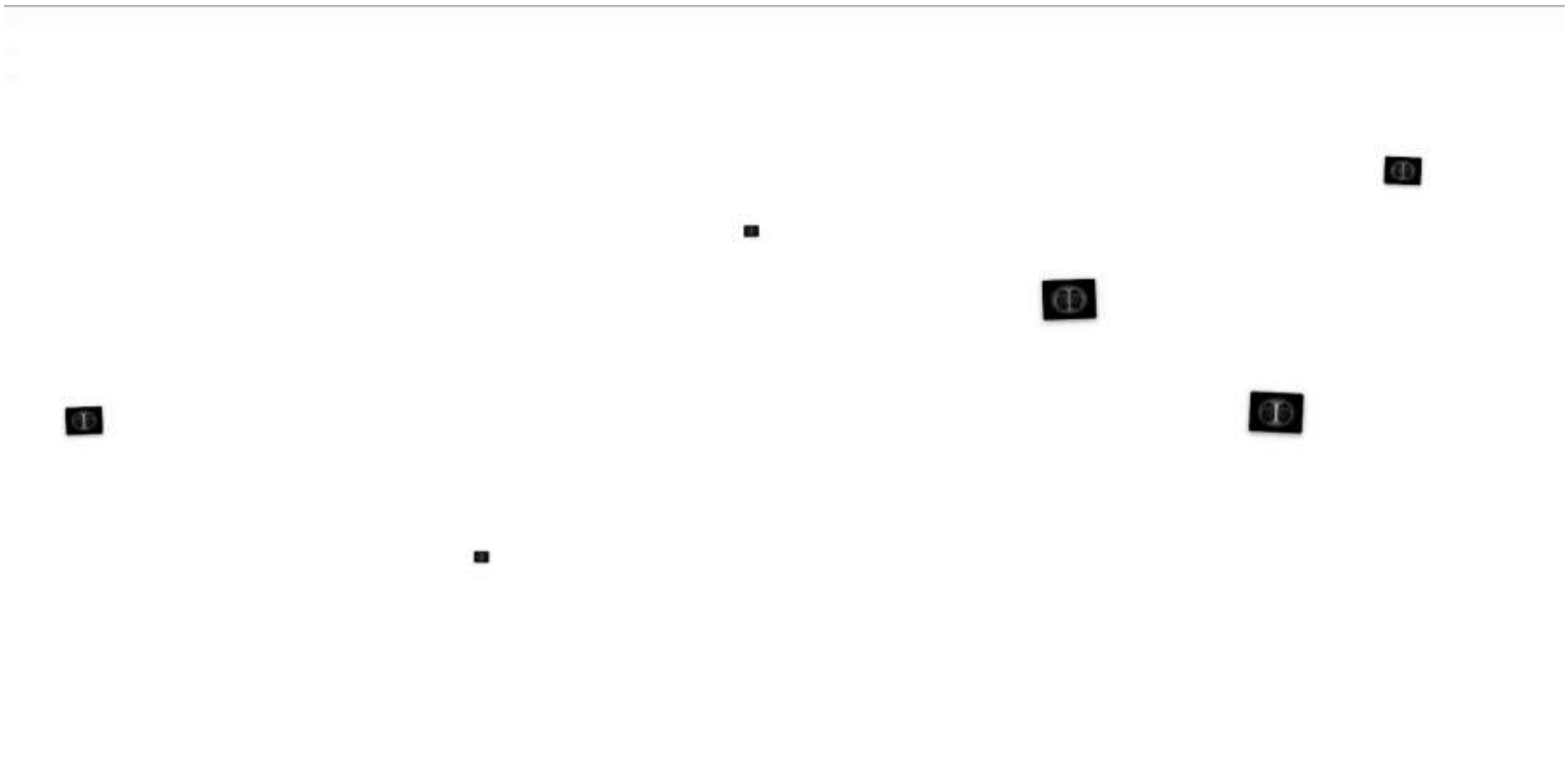
Dockerfile

README.md	chore(docs): remove office files (docx, xlsx) ...	2 weeks ago
VERSIYON_REHBERI.md	Initial frontend backend merge	last month
WSL_AKTIFLESTIR.bat	feat(docker): add production dockerization,...	3 weeks ago
__init__.py	Initial frontend backend merge	last month
convert_paths.py	Initial frontend backend merge	last month
dicom_sorting.py	Initial frontend backend merge	last month
docker-compose.yml	feat(docker): add production dockerization,...	3 weeks ago
generate_dummy_json.py	Initial frontend backend merge	last month
nest-cli.json	Initial frontend backend merge	last month
package-lock.json	docs(readme): enrich README with PR Lens...	last month
package.json	feat(docker): add production dockerization,...	3 weeks ago
requirements-etl.txt	feat(viewer-migration-archive): DICOM 512...	last month
tsconfig.build.json	Initial frontend backend merge	last month
tsconfig.json	Initial frontend backend merge	last month
version.py	Initial frontend backend merge	last month

Sybil UI — Akciğer BT İnceleme Platformu

Akciğer kanseri erken teşhisinde Sybil derin öğrenme risk tahminlerini klinik arayüzle buluşturan, yüksek hızlı tomografi incelemesi ve hekim doğrulama akışları sunan radyolojik karar destek platformu.

NestJS 10 (Fastify) · React 19.0 · DuckDB 1.5.5 OLAP · TypeScript 5.x · Vite 8.2 · Docker Ready | Dual-OS · Archify Interactive Maps
Tests 26 Suites | 179 Passed · ESLint 0 Debt | Clean · PR Lens Architecture Mapped · License MIT



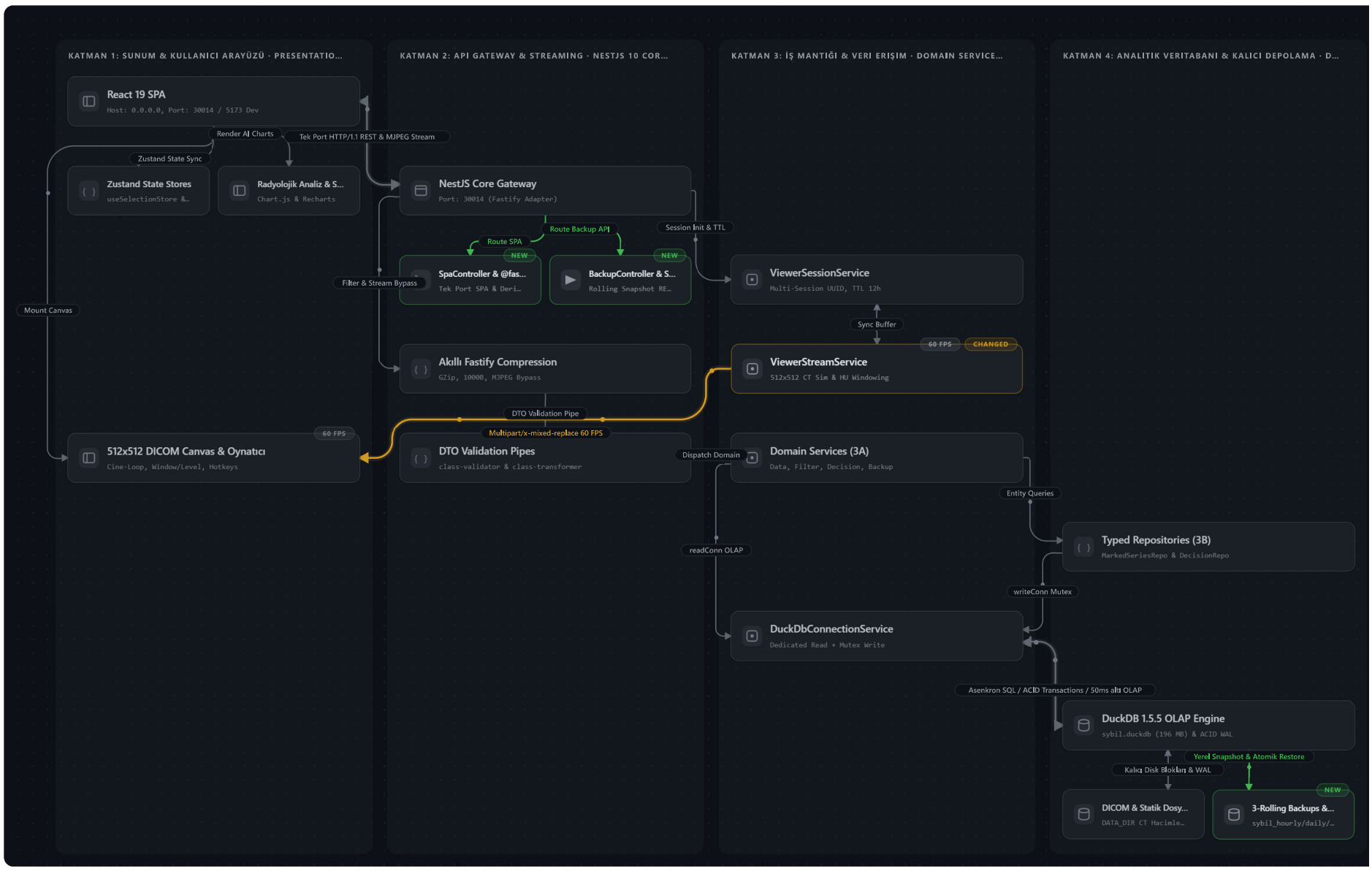
🎬 **Sybil UI Canlı Arayüz Demosu** | *Yüksek çözünürlüklü orijinal kaydı açmak/indirmek için **buraya tıklayın (.mp4)***

1. Platform Mimarisi · 2. Veri Akışları · 3. Yedekleme & Kurtarma · 4. Docker & Ağ · 5. Kurulum · 6. Testler
7. Performans · 8. Archify & PR Lens · 9. UML Şemaları · 10. Dizin Yapısı · 11. Dokümantasyon

1. 🏛️ Uçtan Uca Platform Mimarisi (End-to-End Platform Architecture)

Sybil UI, katı sorumluluk ayrımı (**Separation of Concerns**), tip güvenliği ve sıfır veri sızıntısı ilkelerine dayalı 4 katmanlı uçtan uca bir platform mimarisi üzerinde çalışır:

> **Canlı Veri Akışı & Modül Sınırları:** React 19 SPA sunum katmanından Fastify Gateway'e, Domain Servislerine ve DuckDB 1.5.5 OLAP motoruna kadar tüm modül, servis ve veri yolları arasındaki gerçek zamanlı mimari ve veri trafiği.
> 🟢 **Yeni:** `SpaController` , `BackupController` , `BackupService` , `BackupSchedulerService` | 🟡 **Değişen:** `ViewerStreamService` (60 FPS MJPEG) & `DecisionService` | 🔵 **Canlı Akış:** Gerçek zamanlı soket ve REST veri paketleri
> 🌐 Tam Ekranda Aç (prlens.dev)



▶ *Statik Mermaid Akış Şemasını Görüntüle*

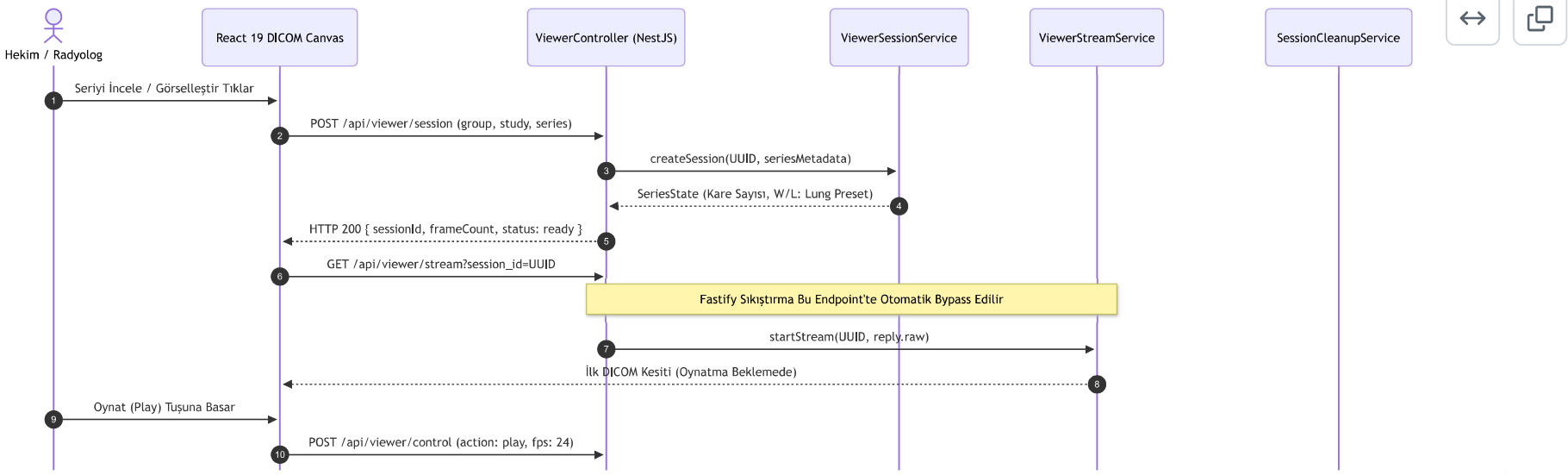
▶ 📋 **Terminal Dostu Katman Özeti (Metin / ASCII Görünümü)**

2. 🎬 Çekirdek Veri Akışları ve Sistem Dinamikleri

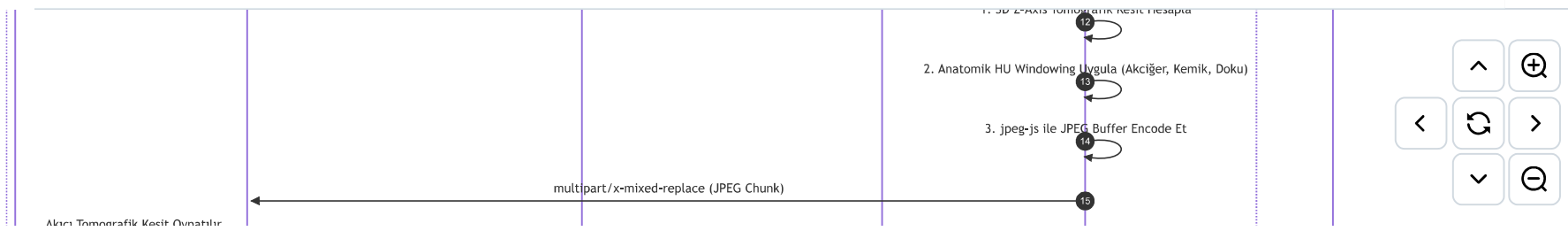
Sybil UI, hekim arayüzünden analitik veritabanına kadar tüm işlemleri katı sorumluluk ayrımı ve asenkron veri hatlarıyla yönetir. Sistemdeki üç ana veri hattı aşağıda detaylandırılmıştır:

2.1 🎞️ 60 FPS DICOM Video Akışı & Cine-Loop Akış Diyagramı

Seri seçildiğinde ayrı bir `/frame` HTTP isteğine gerek kalmaksızın akış oturumu üzerinden ilk kesit anında ekrana gelir; Play tuşuyla kesintisiz 60 FPS video akışı başlar:

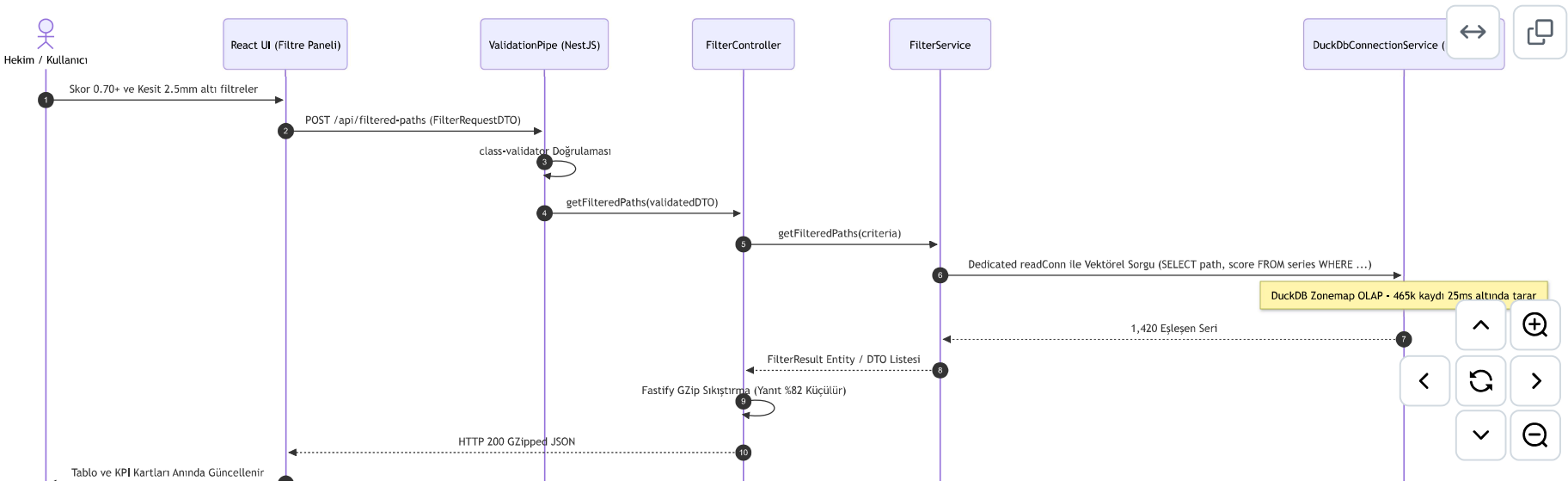


README



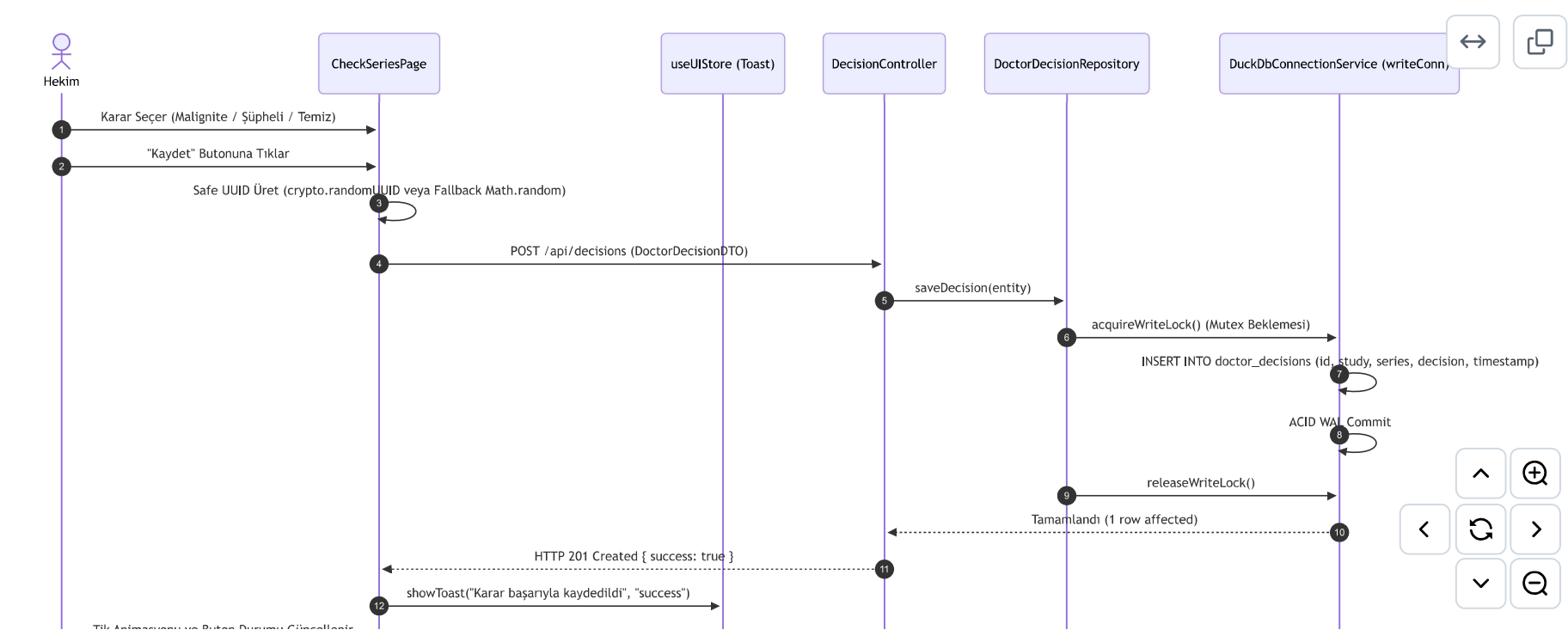
2.2 ⚡ 465.000 Serilik Dinamik Filtreleme ve Fastify GZip Akışı

DuckDB zonemap indeksleri sayesinde 465 bin seri üzerinde karmaşık sorgular **18 — 42 ms** aralığında tamamlanır; büyük JSON yanıtları Fastify GZip ile **%82 küçültülür**:



2.3 📋 Hekim Karar Kayıt ve Atomik ACID Audit Log Akışı

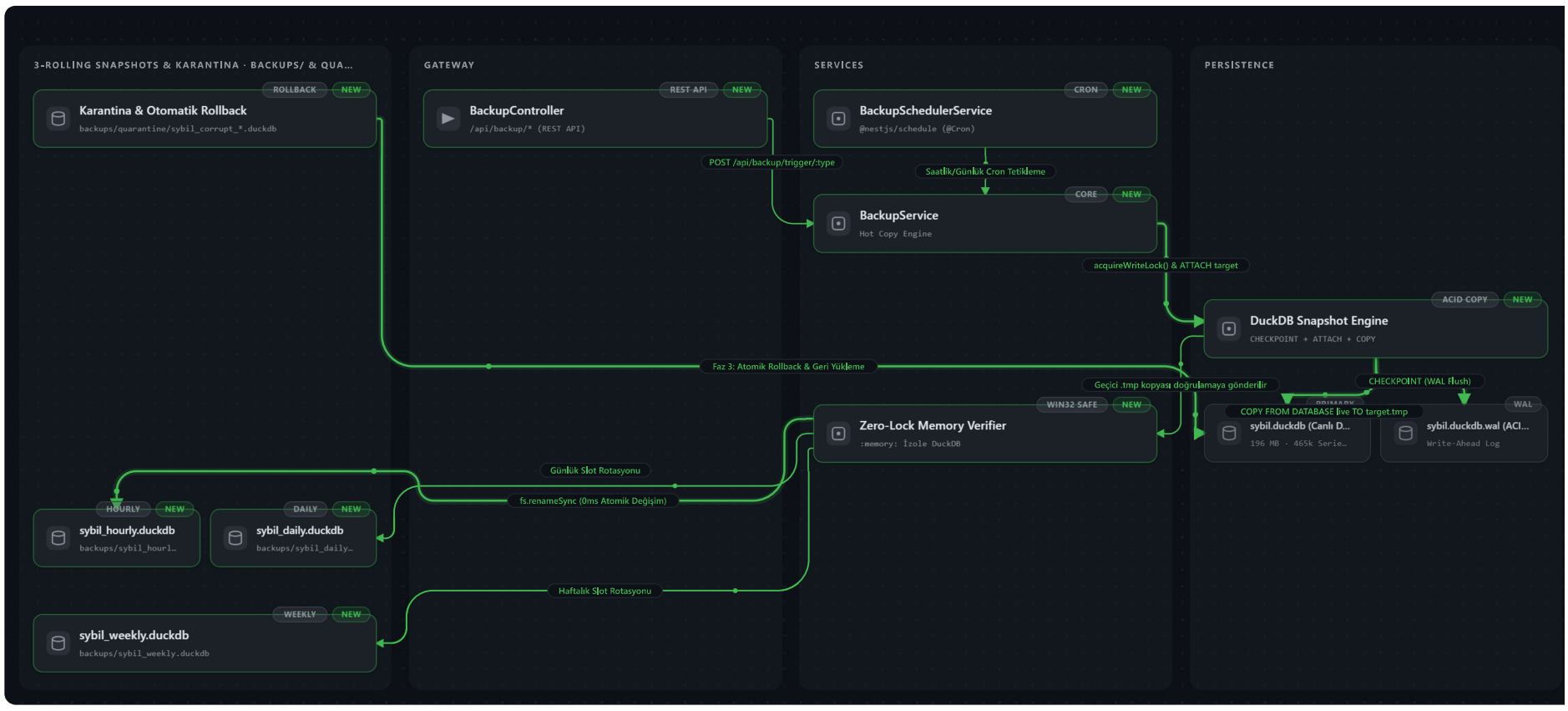
Hekimin klinik kararı, evrensel güvenli UUID fallback mekanizması ve DuckDB yazma mutex'iyle korunan ACID WAL audit tablosuna atomik olarak işlenir:



3. 🔄 Katman 4: DuckDB 3-Rolling Backup ve Felaket Kurtarma Sistemi

Sybil UI; hekim kararları (`doctor_decisions`), işaretlenen seriler (`marked_series`) ve DICOM fihristinin bulunduğu `sybil.duckdb` veritabanının sıfır veri kaybı ve tam felaket kurtarma güvencesi için **4. Katman DuckDB 3-Rolling Backup & Disaster Recovery** mimarisiyle donatılmıştır.

> **Canlı Yedekleme & Afet Kurtarma:** Canlı DuckDB motorundan WAL CHECKPOINT flush'a, yerel ATTACH/COPY snapshot motoruna, izole `:memory:` bütünlük denetimine ve 3-Rolling döner depolamaya (`sybil_hourly` , `sybil_daily` , `sybil_weekly` + `quarantine/`) kadar tüm veri güvenlik hatları.
> 🟢 **Yeni:** `BackupService` , `BackupSchedulerService` & `Volume: sybil-backups` | 🟡 **Değişen:** `restore-db.ts` (CLI 3-Fazlı DR) | 🔵 **Canlı Akış:** Atomik `fs.renameSync` ve otomatik karantina rollback
> 🌐 [Tam Ekranda Aç (prlens.dev)]



▶ *Statik Mermaid Akış Şemasını Görüntüle*

3.1 🛡️ Temel Mimari Prensipleri ve Güvenlik Mekanizmaları

1. **Sabit 3-Rolling Slot Rotasyonu (Sıfır Disk Şişmesi):**
   - Disk üzerinde tarihlere göre sınırsız dosya birikimi engellenmiştir.
   - Yalnızca tam olarak 3 adet döner yedek dosyası tutulur:
     - `backups/sybil_hourly.duckdb` : Her saat başı güncellenir (`0 * * * *`).
     - `backups/sybil_daily.duckdb` : Her gece 00:00'da güncellenir (`0 0 * * *`).
     - `backups/sybil_weekly.duckdb` : Her Pazar 00:00'da güncellenir (`0 0 * * 0`).
   - Sunucu başlangıcında 30 saniye gecikmeyle başlangıç kontrolü (`runStartupCheck`) çalışır ve eksik slotlar anında tamamlanır.
2. **Tamamen Yerel DuckDB Kopya (No Parquet / No Cloud / No Second DB):**
   - S3/Cloud, harici veritabanı veya Parquet dışa aktarımı yoktur.
   - Tüm işlem aynı fiziksel disk üzerinde, doğrudan DuckDB'nin yerel ikili formatında gerçekleşir.
3. **Canlı Snapshot Bütünlüğü (CHECKPOINT + ATTACH + COPY FROM DATABASE):**
   - Ham işletim sistemi dosya kopyalaması (`fs.copyFileSync`) çalışırken yazma olan dosyalarda bozulmaya yol açabileceği için kullanılmaz.
   - `DuckDbConnectionService` önce canlı bağlantı üzerinde `CHECKPOINT` çalıştırarak WAL dosyalarını diske gömer; ardından geçici hedef dosyayı `ATTACH` edip `COPY FROM DATABASE ... TO ...` komutuyla tutarlı ve atomik bir kopya üretir.
4. **İki Aşamalı Atomik Değişim (Atomic Temp Rename):**
   - Yedek önce geçici `.tmp` uzantısıyla oluşturulur (`sybil_hourly.duckdb.tmp`).
   - Dosya doğrulanıp onaylanmadan asıl döner dosyanın üzerine yazılmaz. Doğrulama başarılı olunca `fs.renameSync` ile 0 milisaniyede atomik yer değiştirme yapılır.
5. **Windows EBUSY Handle-Locking Koruması (Zero-Lock Memory Verification):**
   - Windows Win32 dosya kilitleme mimarisinde DuckDB C++ Node.js binding'i dosya kapatılsa bile handle'ı GC çalışana kadar tutabilir.
   - Doğrulama, izole bir `:memory:` DuckDB örneği üzerinde `ATTACH 'path' AS v (READ_ONLY)` ve test sonrası `DETACH v` ile gerçekleştirilir. Bu sayede dosya tanıtıcısı (file handle) anında serbest kalır ve `EBUSY` hatası olmaksızın atomik dosya işlemleri tamamlanır.
6. **Eşzamanlılık Koruması (Concurrency Mutex Lock):**

- Aynı anda birden fazla cron tetiklenmesi veya yedekleme esnasında geri yükleme çağrılması `isBackupRunning` ve `isRestoreRunning` kilitleriyle engellenir.

3.2 🌐 Operasyonel REST API (BackupController)

Yedekleme durumunu izlemek ve anlık yedek alma/doğrulama operasyonları için HTTP API sağlanır:

Metot	Uç Nokta	Açıklama	Örnek Yanıt / İstek
`POST`	`/api/backup/trigger/:type`	Belirtilen slota anlık yedek alır (`hourly` , `daily` , `weekly`)	`curl -X POST http://localhost:30014/api/backup/trigger/hourly`
`GET`	`/api/backup/list`	3 rolling snapshot durumunu ve karantina dosyalarını listeler	`curl http://localhost:30014/api/backup/list`
`GET`	`/api/backup/verify`	Tüm yedeklerin SQL bütünlük raporunu döner	`curl http://localhost:30014/api/backup/verify`

3.3 🚨 3 Fazlı Felaket Kurtarma Protokolü (Disaster Recovery)

Veritabanı hasarı veya acil geri dönüş durumunda `RestoreService` ve CLI aracı 3 fazlı güvenli protokol uygular:

1. **Faz 1 — Aday Yedek Doğrulaması:** Geri yüklenecek yedek dosyasının varlığı, boyutu (> 1 KB), DuckDB başlığı, fiziksel tabloları (`series` , `marked_series` , `doctor_decisions`) ve veri okunabilirliği kontrol edilir. Aday bozuksa işlem anında iptal edilir, canlı sisteme dokunulmaz.
2. **Faz 2 — Canlı Veritabanının Karantinaya Alınması:** Aktif bağlantılar güvenle kapatılır (`closeConnections`). Mevcut aktif `sybil.duckdb` asla doğrudan silinmez; zaman damgasıyla `backups/quarantine/sybil_corrupt_<timestamp>.duckdb` konumuna arşivlenir.
3. **Faz 3 — Atomik Yer Değiştirme ve Otomatik Rollback:** Doğrulanmış aday yedekten geçici kopya oluşturulur ve `data/duckdb/sybil.duckdb` yerine atomik taşınır. Bağlantılar yeniden açılır (`reopenConnections`). Herhangi bir aşamada beklenmeyen hata oluşursa, sistem karantinadaki orijinal veritabanını otomatik olarak yerine koyar (**Rollback**) ve sistemi güvenli duruma döndürür.

3.4 💻 CLI Geri Yükleme & Yedek Denetim Aracı (`scripts/restore-db.ts`)

NestJS uygulamasından bağımsız olarak terminal üzerinden tek komutla derin bütünlük doğrulaması, yedek listeleme ve acil kurtarma gerçekleştirilebilir:

```
# 1. Tüm rolling yedeklerin ve birincil veritabanının derin bütünlük denetimi:
npm run db:verify

# 2. Mevcut 3-Rolling yedekleri ve karantina arşivlerini listeleme:
npm run db:restore -- --list

# 3. En son saatlik yedekten (sybil_hourly.duckdb) canlı veritabanını kurtarma:
npm run db:restore -- --type=hourly

# 4. Günlük veya haftalık yedekten geri yükleme:
npm run db:restore -- --type=daily
npm run db:restore -- --type=weekly

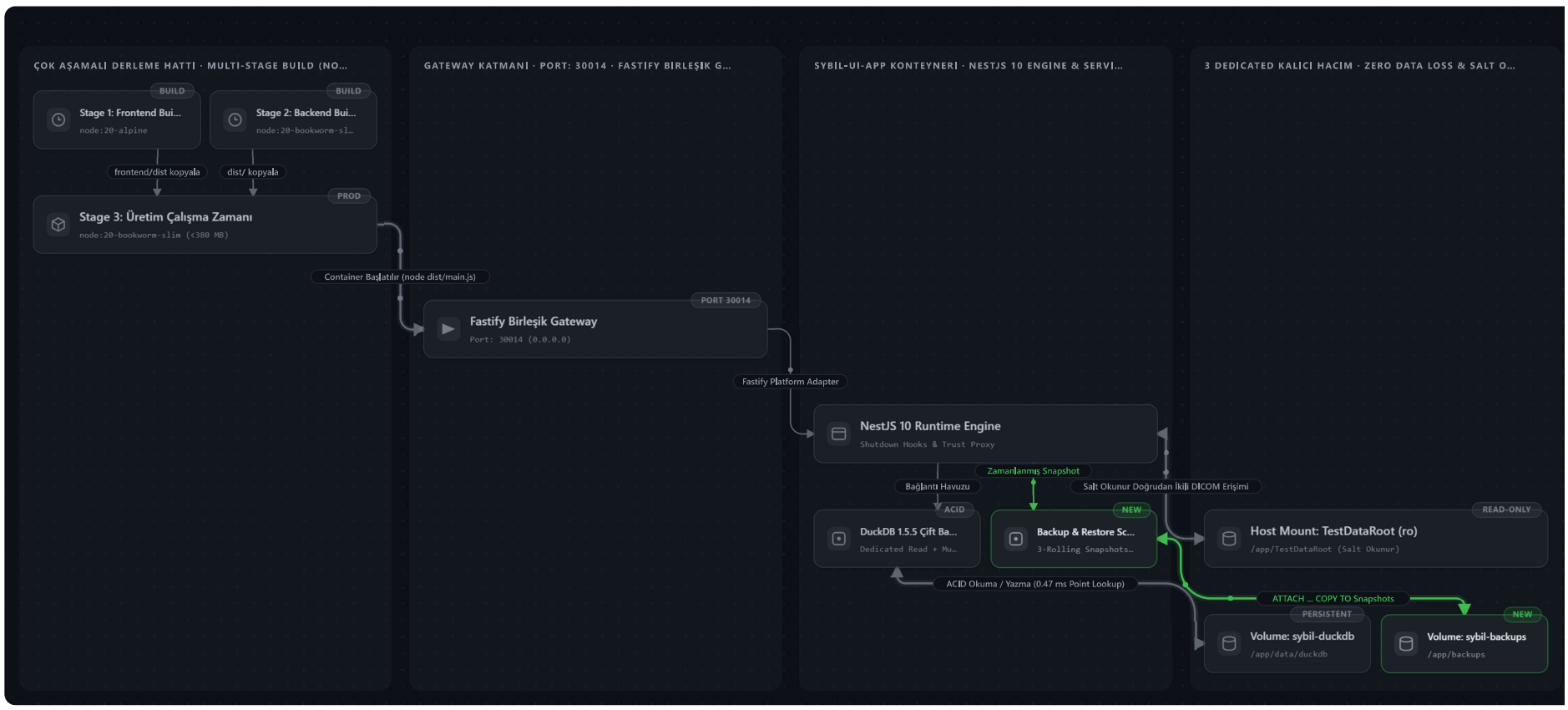
# 5. Özel bir yedek dosyasından canlı veritabanını kurtarma:
npm run db:restore -- --source=backups/sybil_daily.duckdb
```

> **Tip**
>
> Ayrıntılı kriz müdahale senaryoları, SLA taahhütleri (RPO $\leq$ 1h, RTO $\leq$ 2s), olay şiddet matrisi ve karantina yaşam döngüsü için [Felaket Kurtarma El Kitabı'na (docs/DISASTER_RECOVERY_RUNBOOK.md)] bakın.

4. 🐋 Docker Konteyner ve Ağ Dağıtım Topolojisi

Sybil UI v2.0, mikroservis karmaşasını ve harici bağımlılıkları tamamen ortadan kaldıran **Tek Sunucu Birleşik Konteyner (Unified Container)** mimarisiyle donatılmıştır. Çok aşamalı (multi-stage) Docker yapısı sayesinde hem React 19 SPA hem de NestJS Fastify backend tek bir `node:20-bookworm-slim` imajında birleştirilir.

> **Canlı Veri Akışı & Konteyner Depolama:** Çok aşamalı Docker imaj derleme hattından Fastify birleşik gateway'e, NestJS runtime engine'e ve 3 dedicated kalıcı hacme (`sybil-duckdb` , `sybil-backups` , `TestDataRoot:ro`) kadar tüm veri yolları arasındaki sıfır veri kayıplı üretim mimarisi.
> 🟢 **Yeni:** `backup-scheduler` (3-Slot Cron) & `Volume: sybil-backups` | 🟡 **Değişen:** `viewer-stream` (60 FPS MJPEG) | 🔵 **Canlı Akış:** ACID WAL transaction ve doğrudan ikili DICOM akışı
> 🌐 [Tam Ekranda Aç (prlens.dev)]



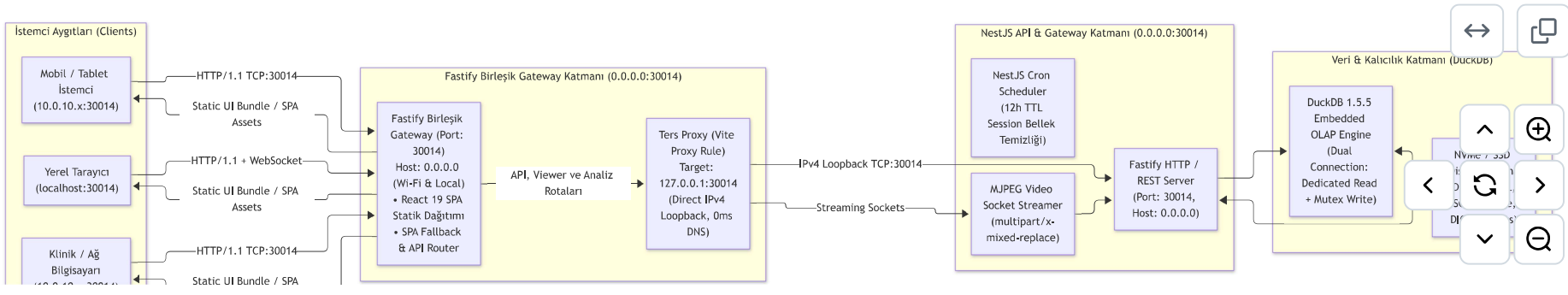
▶ *Statik Mermaid Akış Şemasını Görüntüle*

4.1 🛡️ Docker Güvenlik ve Mimari Prensipleri

1. **3 Dedicated Kalıcı Depolama Alanı (Dedicated Volumes):**
   - `sybil-duckdb` : Veritabanı ve transaction logları (`sybil.duckdb` ve `sybil.duckdb.wal`) saklanır; konteyner silinse dahi veriler korunur.
   - `sybil-backups` : Otomatik saatlik/günlük/haftalık yedekler ve karantina snapshotları saklanır.
   - `TestDataRoot:ro` : DICOM CT görüntüleri salt okunur (`:ro`) olarak bağlanır; konteyner içinden silinme veya bozulma riski %0'dır.
2. **Konteyner İzolasyonu & Non-Root Kullanıcı:**
   - Uygulama, Docker içinde root yetkileriyle değil, standart `node` (UID/GID 1000) kullanıcısı ile çalışır.
3. **Graceful Shutdown (Sıfır Veri Kaybı):**
   - `app.enableShutdownHooks()` devrededir. Docker `SIGTERM` / `SIGINT` gönderdiğinde DuckDB WAL diske flush edilir, açık kilitler serbest bırakılır ve veri kaybı önlenir.
4. **Yerel Node.js Healthcheck:**
   - Harici `curl` veya `wget` paketine gerek duyulmadan, doğrudan saf Node.js scripti (`http.get('/health')`) ile 30 saniyede bir konteyne sağlığı denetlenir.

4.2 🌐 Network & Dağıtım Topolojisi (Dual Connection & Yerel Ağ)

Sistem, hem yerel geliştirme ortamında (`localhost`) hem de yerel ağdaki (LAN Wi-Fi / IP) hekim iş istasyonlarında kesintisiz çalışacak şekilde yapılandırılmıştır:



> 📘 **Ayrıntılı Kılavuz:** Windows WSL 2 konfigürasyonu, ortam değişkenleri ve felaket senaryoları için [Docker & Konteyner Kılavuzu'na (docs/DOCKER_GUIDE.md)] bakın.

5. ⚙️ Kurulum ve Çalıştırma Kılavuzu

5.1 🐳 Docker ile Tek Komutla Kurulum ve Çalıştırma (Önerilen / Production-Ready)

Docker ortamında çalıştırmak, tüm bağımlılıkları izole eder ve 3 Dedicated Volume güvencesiyle verilerinizi korur.

1. **Ortam Dosyasını Hazırlayın:**

```
cp .env.docker.example .env
```

(Windows CMD: `copy .env.docker.example .env`)

2. **Windows Kullanıcıları İçin WSL 2 Kontrolü:** Sisteminizde WSL 2 henüz kurulu değilse, yönetici olarak tek tıkla kurabilirsiniz:

```
WSL_AKTIFLESTIR.bat
```

3. **Konteyneri Derleyin ve Başlatın:**

```
docker compose up -d --build
```

4. **Durum ve Log Kontrolü:**

```
docker compose ps
docker compose logs -f sybil-ui
```

5. **Tarayıcıda Açın:**
   - **Uygulama Arayüzü:** `http://localhost:30014`
   - **Sağlık Denetimi:** `http://localhost:30014/health`

5.2 💻 Klasik Yerel Geliştirme Ortamı (Development Mode)

Docker yerine doğrudan makinenizde geliştirmek isterseniz:

A) Bağımlılıkların Yüklenmesi:

- **Backend Bağımlılıkları:**

```
npm install
```

- **Frontend Bağımlılıkları:**

```
cd frontend
npm install
cd ..
```

- **Python Veritabanı & Analitik Bağımlılıkları:**

```
python -m pip install -r requirements.txt
```

B) Veritabanının Derlenmesi (`sybil.duckdb` Materialization):

> ⚠️ **Önemli:** `data/duckdb/sybil.duckdb` dosyası `.gitignore` kapsamındadır ve **repoya dahil edilmez**. Temiz bir kurulumda sunucuları başlatmadan önce bu adım bir kez çalıştırılmalıdır.

```
python scripts/build_sybil_duckdb.py
```

Script; JSON indekslerini okur, `series`, `marked_series` ve `doctor_decisions` tablolarını oluşturur ve **465.480 seriyi** ZSTD sıkıştırmasıyla DuckDB'ye aktarır.

C) Geliştirme Sunucularının Başlatılması:

- **Terminal 1: NestJS + Fastify Backend Gateway (Port: 30014)**

```
npm run start:dev
```

- **Terminal 2: React 19 + Vite Frontend (Port: 5173)**

```
cd frontend
npm run dev
```

D) Tek Sunucu Üretim Derlemesi:

```
npm run build
npm run start:prod
```

(Sunucu `http://0.0.0.0:30014` adresinde dinlemeye başlar. Hem React 19 SPA statik dosyaları hem de `/api/*` uç noktaları çakışmasız olarak aynı porttan servis edilir).

6. 🧪 Otomatik Test Süitleri ve Kalite Güvencesi

Tüm testler canlı DuckDB veritabanı ve NestJS Gateway üzerinde koşturulabilir:

- **Tüm Test Süitleri (26 Test Süiti / 179 Test - %100 Başarılı):**

```
npm test
```

- **Faz 1: DuckDB Derin Bütünlük, Denetim ve Felaket Kurtarma Testleri (12 Test):**

```
npx jest test/faz1-recovery-integrity.spec.ts --runInBand
```

- **DuckDB Katman 4 Yedekleme & Kurtarma Testleri (Task 2-7, 37 Test):**

```
npx jest test/task7-backup-e2e.spec.ts --runInBand
```

- **Güvenlik Başlıkları & CORS Testi (GATE-015):**

```
npx jest test/gate015-security-cors-helmet.spec.ts
```

- **API Parite Testi (GATE-011):**

```
npx jest test/gate011-api-parity.spec.ts
```

- **DuckDB Davranışsal Parite Testi (GATE-014):**

```
npx jest test/gate014-duckdb-parity.spec.ts
```

- **Performans, Compression ve Concurrency Benchmark (GATE-012):**

```
npx jest test/gate012-benchmarks.spec.ts
```

- **Frontend Lint Kontrolü (0 Hata / 0 Uyarı):**

```
npm --prefix frontend run lint
```

7. 📊 Performans Karşılaştırma Matrisi

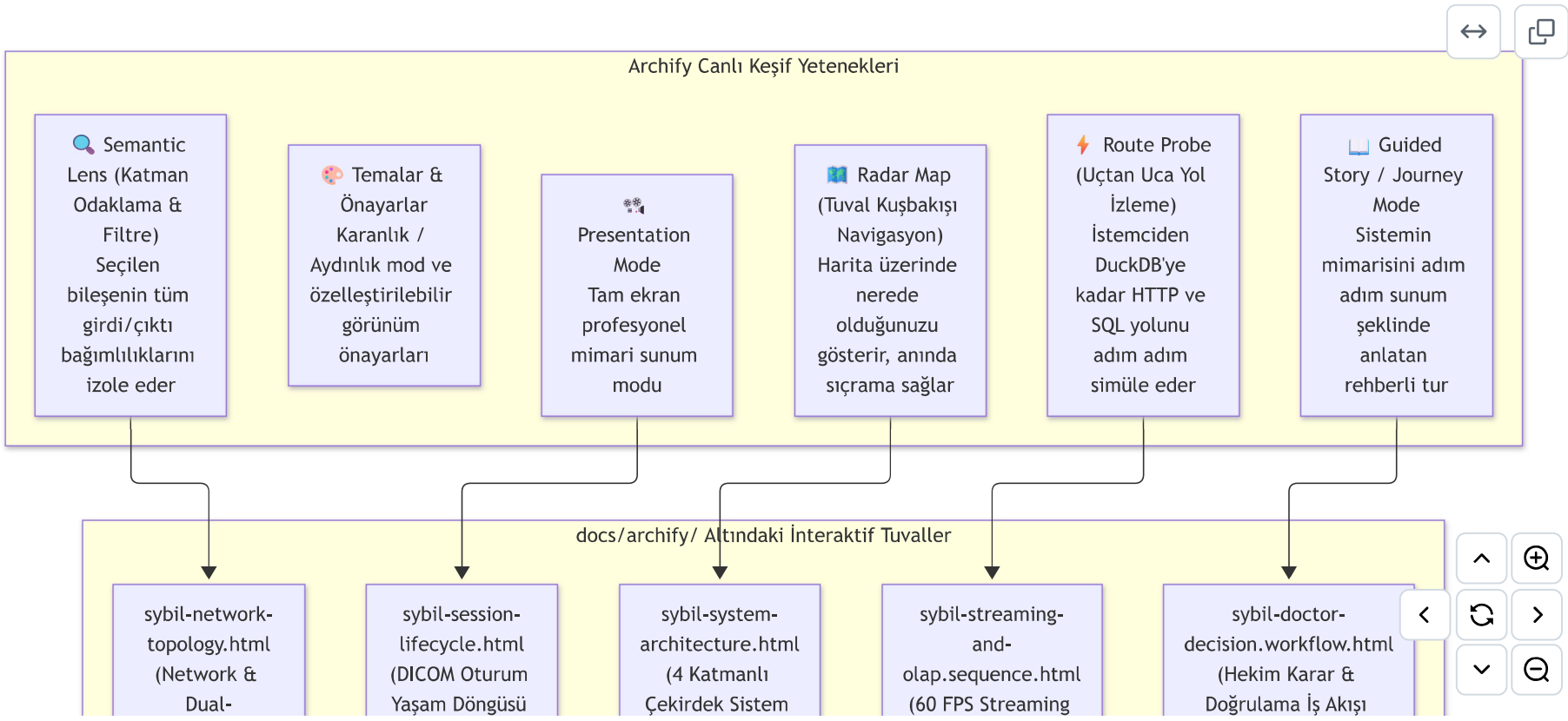
Özellik / Metrik	Eski FastAPI + Ham JSON Mimarisi	Yeni DuckDB 1.5.5 + NestJS 10 Mimarisi	İyileşme / Kazanım
465.000 Seri Filtreleme	1.800 ms - 3.200 ms	**18 ms - 42 ms**	**~75x Kat Hızlanma (OLAP Vektörel)**
Tekil Seri Sorgusu (Point Lookup)	85 ms - 220 ms	**0.47 ms**	**~200x Kat Hızlanma**
Veri Boyutu (Disk Alanı)	2.14 GB (Ham JSON İndeksleri)	**196.26 MB (DuckDB ZSTD)**	**%91 Disk Tasarrufu**
Bellek Tüketimi (RAM)	3.8 GB - 4.5 GB (JSON In-Memory)	**410 MB - 680 MB Sabit**	**%84 Bellek Tasarrufu**
DICOM Video Akış Hızı	15 - 24 FPS (Tarayıcı Donmaları)	**60 FPS Kilitli Akıcılık**	**Tam Akıcı Cine-Loop Deneyimi**

Özellik / Metrik	Eski FastAPI + Ham JSON Mimarisi	Yeni DuckDB 1.5.5 + NestJS 10 Mimarisi	İyileşme / Kazanım
JSON Ağ Yükü (Compression)	4.8 MB (Ham JSON Payload)	**850 KB (Fastify Akıllı GZip)**	**%82 Ağ Bant Genişliği Tasarrufu**
Konteyner ve Dağıtım	Çoklu Terminal & Bağımlılık Çakışması	**Multi-Stage Docker (3 Dedicated Volume)**	**Taşınabilir, İzole & 0-Config Dağıtım**
Afet Kurtarma & RPO Güvencesi	Manuel Dosya Kopyalama / Yok	**Otomatik 3-Rolling Snapshots (RPO ≤ 1h)**	**Sıfır Veri Kaybı (Zero Data Loss)**

8. 🗺️ Archify, PR Lens ve İnteraktif Mimari Haritaları

8.1 🗺️ Archify İnteraktif Mimari ve Akış Haritası

Sybil-UI mimarisi, statik resimlerin ve şemaların ötesine geçerek **Archify** etkileşimli mimari tuval motoru ile tam entegre çalışır. Archify; 4 katmanlı sistem bileşenlerini, veri akışını ve hekim karar zincirini tarayıcınızda canlı bir harita gibi keşfetmenizi sağlar.



8.2 ⌨️ Archify Klavye Kısayolları (Hotkeys Cheatsheet)

Archify tuvalleri açıkken aşağıdaki klavye tuşları ile doğrudan etkileşime geçebilirsiniz:

- `F` : **Finder** — Sistemdeki herhangi bir modül, servis, tablo veya API rotasını anında arayın.
- `M` : **Map / Radar** — Kuşbakışı radar mini-haritasını açıp kapatın.
- `L` : **Semantic Lens** — Seçili bileşene odaklanıp çevresindeki gereksiz gürültüyü gizleyin.
- `R` : **Route Probe** — Uçtan uca ağ ve veri yolunu canlı simüle edin.
- `P` : **Presentation Mode** — Tam ekran profesyonel sunum moduna geçin.
- `S` : **Preset Cycle** — Mimari renk ve detay önayarları arasında geçiş yapın.
- `T` : **Theme Toggle** — Koyu (Dark) ve Aydınlık (Light) tema arasında geçiş yapın.
- `Esc` : Açık olan modalleri veya odaklamayı sıfırlayın.

> 🌐 **Tüm Tuvalleri Tek Ekranda Açın:** Tüm bu tuvalleri iframe'ler ve hızlı geçiş butonlarıyla tek bir sayfada toplayan ana gösterge paneli: docs/SYSTEM_ARCHITECTURE_AND_DIAGRAMS_ARCHIFY.html

8.3 🔍 PR Lens Entegrasyonu — Görsel PR Denetimi

Sybil UI, kod değişikliklerini ve mimari etkisini çekme isteklerinde (Pull Requests) **otomatik olarak görselleştirmek** için PR Lens entegrasyonu ile donatılmıştır.

Her çekme isteğinde PR Lens, diff'i analiz ederek etkilenen bileşenleri katmanlarına göre renklendirir (**yeşil** yeni, **sarı** değişen, **kırmızı** silinen) ve veri akışını animasyonlu mimari şema olarak PR açıklamasına ekler:

```
Katman 1: React 19 Frontend SPA ⟶ Katman 2: NestJS Gateway ⟶ Katman 3B: Repositories ⟶ Katman 4: DuckDB
```

Yapılandırma (`.github/pr-lens.yml`):

Sybil UI'ın katmanlı mimarisi repository yapılandırma dosyası ile eşleştirilmiştir:

```yaml
schemaVersion: 0.1.1
lenses: [architecture, data-flow]
branding: true

map:
  exclude:
    - "**/*.spec.ts"
    - "**/*.test.ts"
    - "scripts/**"
  lane:
    - match: frontend/**
      lane: "Katman 1: React 19 Frontend SPA"
    - match: src/modules/**
      lane: "Katman 2: NestJS Fastify Gateway"
    - match: src/common/**
      lane: "Katman 2: Ortak Güvenlik & Altyapı"
    - match: src/repositories/**
      lane: "Katman 3B: Typed Repositories & Engine"
    - match: data/duckdb/**
      lane: "Katman 4: DuckDB Embedded OLAP"
```

8.4 🌐 İnteraktif Tuval & Web Mimari Raporları Tablosu

Sistemin tüm mimarisini, Mermaid akışlarını ve ağ topolojisini tarayıcınızda dinamik olarak yakınlaştırıp inceleyebilirsiniz:

Rapor	Format	Açıklama
Archify Sistem Mimarisi & Akış Haritası	`Archify Master Dashboard`	**Tüm sistemin canlı Archify tuvali**, iframe gezinti sekmeleri, klavye kısayolları ve tam ekran sunumu
Archify Çekirdek Sistem Tuvali	`Archify Interactive Canvas`	4 katmanlı çekirdek mimari, Radar Map, Semantic Lens ve katman bağımlılıkları
Archify Ağ Topolojisi Tuvali	`Archify Interactive Canvas`	Dual-Connection, LAN ve istemci ağ dağıtımının canlı radar tuvali
Archify Streaming & OLAP Sequence	`Archify Interactive Canvas`	60 FPS MJPEG görüntüleme ve 465k filtreleme sıralama akışının interaktif haritası
Archify Hekim Karar İş Akışı	`Archify Interactive Canvas`	Hekim kayıt buton durumu, klinik karar modeli ve DuckDB WAL audit log akış tuvali
Archify Oturum Yaşam Döngüsü	`Archify Interactive Canvas`	DICOM görüntüleme oturum yaşam döngüsü ve 12 saatlik bellek tahliye tuvali
Sistem Mimarisi & UML Raporu	`HTML / Mermaid`	Tüm 4 katmanlı mimarinin, sequence ve class diyagramlarının interaktif Mermaid sunumu
Sistem Mimarisi Teknik Dokümanı	`Markdown`	Kapsamlı teknik mimari, Docker volume topolojisi ve UML sınıf açıklamaları
Docker & Konteyner Kılavuzu	`Markdown`	Dual-OS Docker mimarisi, WSL2 entegrasyonu, persistent volumes ve operasyonel kılavuz
Archify / PR Lens DuckDB 3-Rolling Backup	`PR Lens Interactive Canvas`	DuckDB 1.5.5 ATTACH/COPY TO snapshot, izole :memory: doğrulama ve 3 fazlı afet kurtarma tuvali
Uçtan Uca Mimari Akış ve Sequence	`Markdown`	Hiyerarşik gezinme, filtreleme, streaming ve hekim kararları detaylı sequence raporu

9. 🔍 Derinlemesine Teknik UML ve Durum Şemaları

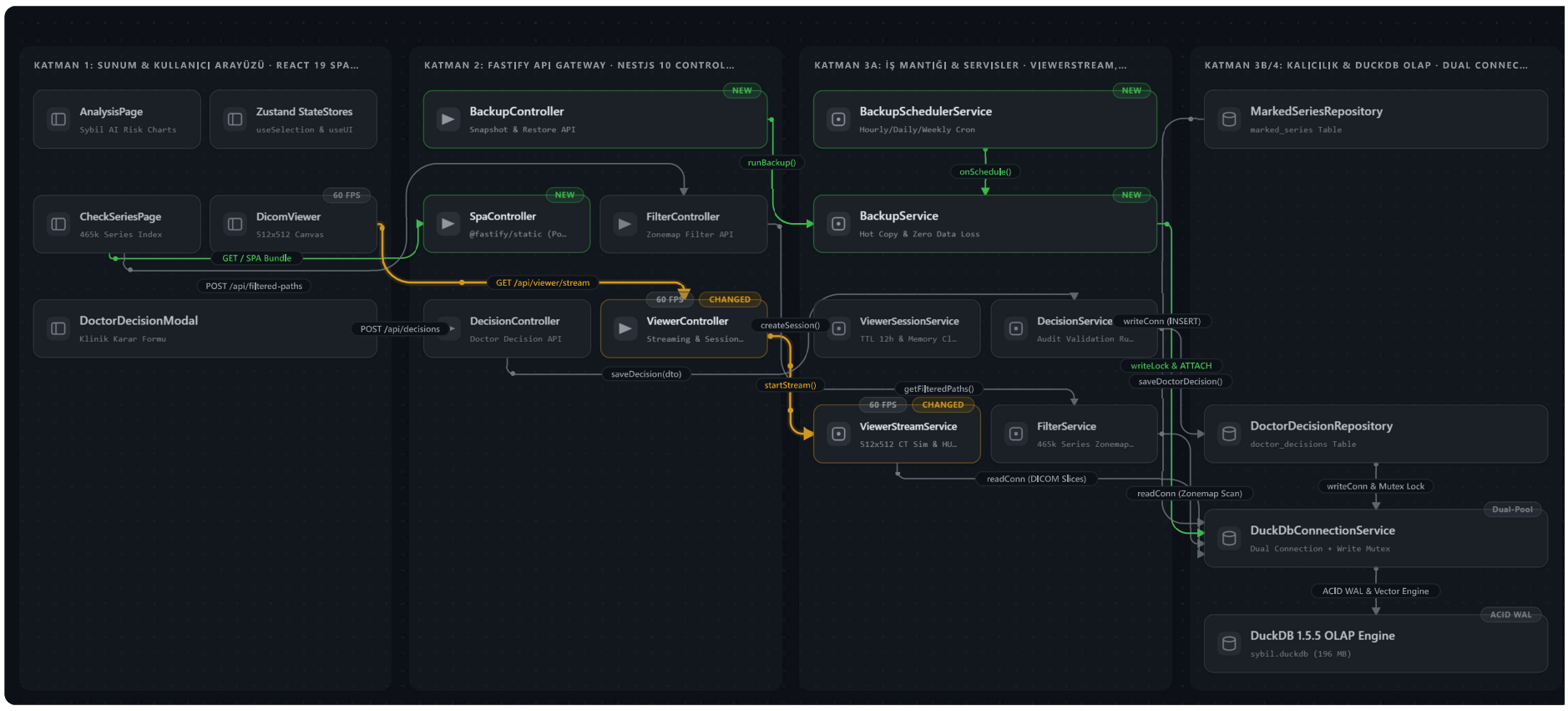
Geliştiriciler ve mimari denetçiler için detaylandırılmış UML şemaları ve durum makineleri:

9.1 📐 UML Sınıf Diyagramı (Class Diagram & Tip İlişkileri)

▶ *UML Sınıf Diyagramını Genişlet*

9.2 📦 Kurumsal Katman & Paket Mimarisi (Architecture View)

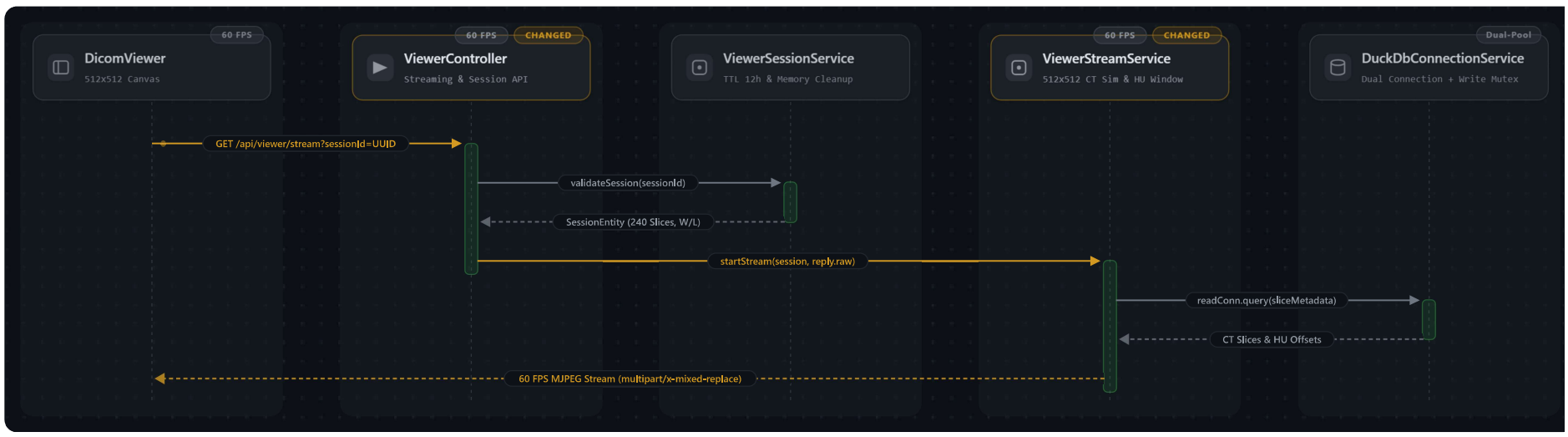
> **Canlı Veri Akışı & Modül Sınırları:** React 19 SPA presentation katmanından Fastify Gateway'e, Domain Servislerine ve DuckDB 1.5.5 OLAP motoruna kadar tüm modül, servis ve veri yolları arasındaki gerçek zamanlı mimari ve veri trafiği.
> 🟢 **Yeni:** `SpaController` , `BackupController` , `BackupService` , `BackupSchedulerService` | 🟡 **Değişen:** `ViewerStreamService` (60 FPS MJPEG) & `DecisionService` | 🔵 **Canlı Akış:** Gerçek zamanlı soket ve REST paketleri
> 🌐 Tam Ekranda Aç (prlens.dev)



▶ *Statik Mermaid Kodunu Görüntüle*

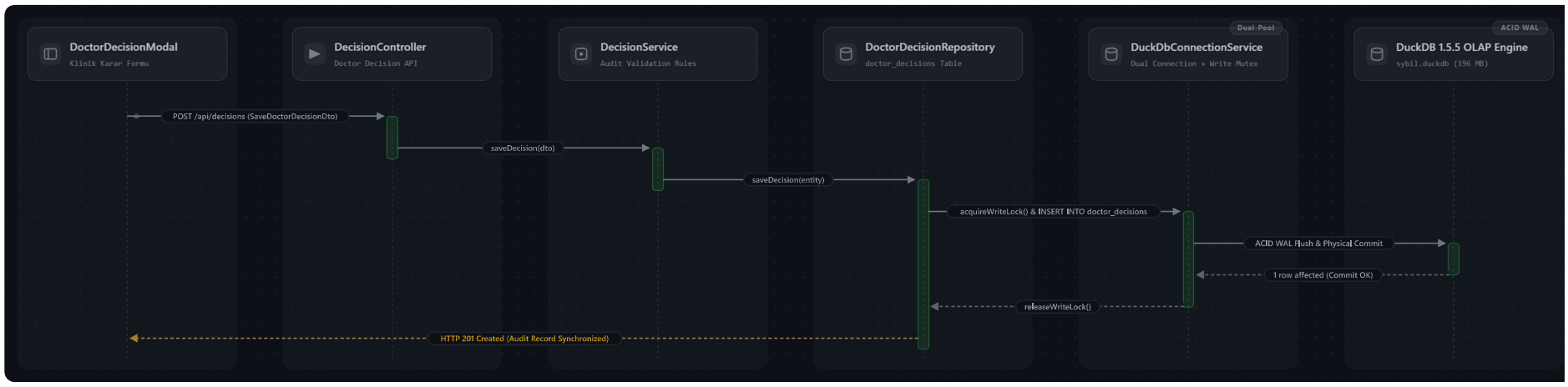
9.3 🎞️ DICOM İnceleme & 60 FPS Video Streaming Akışı (Data-Flow Sequence View)

> **60 FPS Video Akışı:** React 19 Viewer Canvas (`DicomViewer`) ile Fastify `ViewerController` , `ViewerSessionService` , `ViewerStreamService` ve DuckDB Dedicated `readConn` arasındaki düşük gecikmeli, kesintisiz MJPEG stream sıralı veri akışı.
> 🎞️ **DICOM Streaming:** `DicomViewer` ➔ `ViewerController` ➔ `ViewerSessionService` ➔ `ViewerStreamService` (readConn) ➔ 60 FPS MJPEG Buffer
> 🌐 Tam Ekranda Aç (prlens.dev)



9.4 ⚖️ Hekim Karar Kayıt & DuckDB WAL Mutex Akışı (Data-Flow Sequence View)

> **ACID Hekim Karar Kaydı:** Hekimin klinik kararının `DoctorDecisionModal` ➔ `DecisionController` ➔ `DecisionService` ➔ `DoctorDecisionRepository` üzerinden alınıp DuckDB Write Mutex (`writeConn`) korumasıyla fiziksel WAL audit loguna atomik kaydedilmesinin sıralı akışı.
> ⚖️ **Karar Kayıt:** `DecisionController` ➔ `DecisionService` ➔ `DoctorDecisionRepository` ➔ DuckDB Write Mutex ➔ ACID WAL Commit
> 🌐 Tam Ekranda Aç (prlens.dev)



9.5 🔄 Oturum ve Buton Durum Makineleri (State Machines)

▶ *Durum Makinelerini Görüntüle*

9.6 🌐 Fiziksel Dağıtım ve Ağ Düğümleri (Deployment Diagram)

▶ *Fiziksel Dağıtım Şemasını Görüntüle*

10. 📁 Proje Dizin Yapısı

```
sybil-ui/
├── Dockerfile                    # Çok Aşamalı (Multi-Stage) Üretim Docker İmajı Tanımı
```